Git

Used For:

- Tracking Code Changes: Monitor modifications in source code to understand project evolution.

- Who Made Changes: Record contributions by author to ensure proper attribution and accountability.

- Coding Collaboration: Facilitate teamwork through tools and practices that enhance communication and code integration.

What does it do?

- Manage Projects with Repositories: Git uses repositories (repos) to store project files and their complete history of changes.

- Clone a Project: You can create a local copy of a remote repository to work on it offline.

- Control and Track Changes: Git allows you to track modifications using staging (preparing changes for commit) and committing (saving changes to the repo).

- Branch and Merge: You can create branches to work on different features or versions of a project, then merge them back into the main branch as needed.

- Pull Latest Version: You can update your local copy with the most recent changes from the main repository.

- Push Local Updates: After making changes locally, you can push those updates back to the main repository.

Working with it:

- Initialize Git: Start by turning a folder into a Git repository using the command git init.

- Hidden Folder Creation: Git creates a hidden .git folder to track all changes within that project folder.

- File Modifications: Any changed, added, or deleted files are marked as modified.

- Staging Changes: Select the modified files you want to prepare (stage) for commitment to the repository.

- Committing Changes: Staged files are committed, creating a permanent snapshot of their current state in the repository.

- View Commit History: Git provides a full history of every commit, allowing you to review past changes.

- Revert Commits: You can revert to any previous commit if needed.

- Efficient Storage: Instead of storing separate copies of every file for each commit, Git tracks the changes made, saving space.

Why Git?

- Widespread Adoption: Over 70% of developers use Git as their version control system.

- Global Collaboration: Developers can collaborate on projects from anywhere in the world, making remote teamwork efficient.

- Project History: Git allows developers to view the complete history of a project, including all changes and contributions.

- Version Reversion: Developers can easily revert to earlier versions of a project, providing flexibility and safety in development.

Github:

- Git vs. GitHub: Git is a version control system used for tracking changes in files. GitHub is a web-based platform that provides tools and hosting services specifically for projects that use Git.

- Tools and Hosting: GitHub offers a variety of tools that enhance collaboration and project management for Git users.

- Largest Source Code Host: GitHub is the world's largest repository for source code and has been owned by Microsoft since 2018.

Git Install

- You can download Git for free from the following website: https://www.git-scm.com/

Using Git with Command Line

- To check if Git is properly installed:

```
git --version
```

Configure Git

```
git config --global user.name "RR"
git config --global user.email "rr@gmail.com"
```

Note:

- Global: Sets username and email for all Git repositories.

- Local: Sets username and email for the current repository only (remove `global`).

Creating Git Folder

mkdir myproject

cd myproject

Explanation:

mkdir makes a new directory.

cd changes the current working directory.

Initialize Git

git init

Git Adding New Files

create a file (index.html) then type

ls

index.html

check git if this file is a part of our repository:

git status

On branch master

No commits yet

Untracked files:

  (use "git add ..." to include in what will be committed)

index.html

nothing added to commit but untracked files present (use "git add" to track)

Note:

In Git, files are either *tracked* (added to the repository) or *untracked* (not added), and untracked files must be staged to be tracked.

Git Staging Environment

git add index.html

The file should be Staged.

Let's check the status:

git status

On branch master

No commits yet

Changes to be committed:

(use "git rm --cached ..." to unstage)

new file: index.html

Git Add More than One File

README.md = A file that describes the repository

Add all files to staging

git add --all

Explanation:

`--all` stages all changes to new, modified, and deleted files.

for checking :

git status

On branch master

No commits yet

Changes to be committed:

(use "git rm --cached ..." to unstage)

new file: README.md

new file: bluestyle.css

new file: index.html

Important

The shorthand command for git add --all is git add -A

Git Commit

Committing in Git tracks changes as "save points" and should include clear messages for clarity.

git commit -m "First release of Hello World!"

[master (root-commit) 221ec6e] First release of Hello World!

3 files changed, 26 insertions(+)

create mode 100644 README.md

create mode 100644 bluestyle.css

create mode 100644 index.html

Explanation:

-m "message"

The commit command finalizes changes with a message; the Staging Environment is committed as "First release of Hello World!"

Git Commit without Stage

Use the `-a` option to directly commit small changes, skipping the staging environment.

git status --short

M index.html

Explanation:

Note: Short status flags are:

?? - Untracked files

A - Files added to stage

M - Modified files

D - Deleted files

We see the file we expected is modified. So let's commit it directly:

git commit -a -m "Updated index.html with a new line"

[master 09f4acd] Updated index.html with a new line

1 file changed, 1 insertion(+)

Check the compact version of the status for repository:

git status --short

Commit the updated files directly, skipping the staging environment:

git commit -a -m "Updated index.html with a new line"

Git Commit Log

You can view a repository's commit history by using the log command.

git log

commit 09f4acd3f8836b7f6fc44ad9e012f82faf861803 (HEAD -> master)

Author: w3schools-test

Date: Fri Mar 26 09:35:54 2021 +0100

    Updated index.html with a new line

commit 221ec6e10aeedbfd02b85264087cd9adc18e4b26

Author: w3schools-test

Date: Fri Mar 26 09:13:07 2021 +0100

    First release of Hello World!

Git Help

Use Git help with `git command -help` for options or `git help --all` for all commands.

Git -help See Options for a Specific Command

git commit -help

usage: git commit [] [--] ...

-q, --quiet suppress summary after successful commit

-v, --verbose show diff in commit message template

Commit message options

-F, --file read message from file

--author override author for commit

--date override date for commit

-m, --message

commit message

-c, --reedit-message

reuse and edit message from specified commit

-C, --reuse-message

reuse message from specified commit

--fixup use autosquash formatted message to fixup specified commit

--squash use autosquash formatted message to squash specified commit

--reset-author the commit is authored by me now (used with -C/-c/--amend)

-s, --signoff add a Signed-off-by trailer

-t, --template

use specified template file

-e, --edit force edit of commit

--cleanup how to strip spaces and #comments from message

--status include status in commit message template

-S, --gpg-sign[=]

GPG sign commit

Commit contents options

-a, --all commit all changed files

-i, --include add specified files to index for commit

--interactive interactively add files

-p, --patch interactively add changes

-o, --only commit only specified files

-n, --no-verify bypass pre-commit and commit-msg hooks

--dry-run show what would be committed

--short show status concisely

--branch show branch information

--ahead-behind compute full ahead/behind values

--porcelain machine-readable output

--long show status in long format (default)

-z, --null terminate entries with NUL

--amend amend previous commit

--no-post-rewrite bypass post-rewrite hook

-u, --untracked-files[=]

show untracked files, optional modes: all, normal, no. (Default: all)

--pathspec-from-file

read pathspec from file

--pathspec-file-nul with --pathspec-from-file, pathspec elements are separated with NUL character

Note:

You can use `--help` instead of `-help` to access the relevant Git manual page.

Git help --all See All Possible Commands

$ git help --all

See 'git help ' to read about a specific subcommand

Main Porcelain Commands

add	Add file contents to the index
am	Apply a series of patches from a mailbox
archive	Create an archive of files from a named tree
bisect	Use binary search to find the commit that introduced a bug
branch	List, create, or delete branches
bundle	Move objects and refs by archive
checkout	Switch branches or restore working tree files
cherry-pick	Apply the changes introduced by some existing commits
citool	Graphical alternative to git-commit
clean	Remove untracked files from the working tree
clone	Clone a repository into a new directory
commit	Record changes to the repository
describe	Give an object a human readable name based on an available ref
diff	Show changes between commits, commit and working tree, etc
fetch	Download objects and refs from another repository
format-patch	Prepare patches for e-mail submission
gc	Cleanup unnecessary files and optimize the local repository
gitk	The Git repository browser
grep	Print lines matching a pattern
gui	A portable graphical interface to Git
init	Create an empty Git repository or reinitialize an existing one
log	Show commit logs
maintenance	Run tasks to optimize Git repository data
merge	Join two or more development histories together

mv Move or rename a file, a directory, or a symlink

notes Add or inspect object notes

pull Fetch from and integrate with another repository or a local branch

push Update remote refs along with associated objects

range-diff Compare two commit ranges (e.g. two versions of a branch)

rebase Reapply commits on top of another base tip

reset Reset current HEAD to the specified state

restore Restore working tree files

revert Revert some existing commits

rm Remove files from the working tree and from the index

shortlog Summarize 'git log' output

show Show various types of objects

sparse-checkout Initialize and modify the sparse-checkout

stash Stash the changes in a dirty working directory away

status Show the working tree status

submodule Initialize, update or inspect submodules

switch Switch branches

tag Create, list, delete or verify a tag object signed with GPG

worktree Manage multiple working trees

Ancillary Commands / Manipulators

config Get and set repository or global options

fast-export Git data exporter

fast-import Backend for fast Git data importers

filter-branch Rewrite branches

mergetool Run merge conflict resolution tools to resolve merge conflicts

pack-refs Pack heads and tags for efficient repository access

prune Prune all unreachable objects from the object database

reflog Manage reflog information

remote Manage set of tracked repositories

repack Pack unpacked objects in a repository

replace Create, list, delete refs to replace objects

Ancillary Commands / Interrogators

annotate Annotate file lines with commit information

blame Show what revision and author last modified each line of a file

bugreport Collect information for user to file a bug report

count-objects Count unpacked number of objects and their disk consumption

difftool Show changes using common diff tools

fsck Verifies the connectivity and validity of the objects in the database

gitweb Git web interface (web frontend to Git repositories)

help Display help information about Git

instaweb Instantly browse your working repository in gitweb

merge-tree Show three-way merge without touching index

rerere Reuse recorded resolution of conflicted merges

show-branch Show branches and their commits

verify-commit Check the GPG signature of commits

verify-tag Check the GPG signature of tags

whatchanged Show logs with difference each commit introduces

Interacting with Others

archimport Import a GNU Arch repository into Git

cvsexportcommit Export a single commit to a CVS checkout

cvsimport Salvage your data out of another SCM people love to hate

cvsserver A CVS server emulator for Git

imap-send Send a collection of patches from stdin to an IMAP folder

p4 Import from and submit to Perforce repositories

quiltimport Applies a quilt patchset onto the current branch

request-pull Generates a summary of pending changes

send-email Send a collection of patches as emails

svn Bidirectional operation between a Subversion repository and Git

Low-level Commands / Manipulators

apply Apply a patch to files and/or to the index

checkout-index Copy files from the index to the working tree

commit-graph Write and verify Git commit-graph files

commit-tree Create a new commit object

hash-object Compute object ID and optionally creates a blob from a file

index-pack Build pack index file for an existing packed archive

merge-file Run a three-way file merge

merge-index Run a merge for files needing merging

mktag Creates a tag object

mktree Build a tree-object from ls-tree formatted text

multi-pack-index Write and verify multi-pack-indexes

pack-objects Create a packed archive of objects

prune-packed Remove extra objects that are already in pack files

read-tree Reads tree information into the index

symbolic-ref Read, modify and delete symbolic refs

unpack-objects Unpack objects from a packed archive

update-index Register file contents in the working tree to the index

update-ref Update the object name stored in a ref safely

write-tree Create a tree object from the current index

Low-level Commands / Interrogators

cat-file Provide content or type and size information for repository objects

cherry Find commits yet to be applied to upstream

diff-files Compares files in the working tree and the index

diff-index Compare a tree to the working tree or index

diff-tree Compares the content and mode of blobs found via two tree objects

for-each-ref Output information on each ref

for-each-repo Run a Git command on a list of repositories

get-tar-commit-id Extract commit ID from an archive created using git-archive

ls-files Show information about files in the index and the working tree

ls-remote List references in a remote repository

ls-tree List the contents of a tree object

merge-base Find as good common ancestors as possible for a merge

name-rev Find symbolic names for given revs

pack-redundant Find redundant pack files

rev-list Lists commit objects in reverse chronological order

rev-parse Pick out and massage parameters

show-index Show packed archive index

show-ref List references in a local repository

unpack-file Creates a temporary file with a blob's contents

var Show a Git logical variable

verify-pack Validate packed Git archive files

Low-level Commands / Syncing Repositories

daemon A really simple server for Git repositories

fetch-pack Receive missing objects from another repository

http-backend Server side implementation of Git over HTTP

send-pack Push objects over Git protocol to another repository

update-server-info Update auxiliary info file to help dumb servers

Low-level Commands / Internal Helpers

check-attr Display gitattributes information

check-ignore Debug gitignore / exclude files

```
  check-mailmap       Show canonical names and email addresses of contacts
  check-ref-format    Ensures that a reference name is well formed
  column              Display data in columns
  credential          Retrieve and store user credentials
  credential-cache    Helper to temporarily store passwords in memory
  credential-store    Helper to store credentials on disk
  fmt-merge-msg       Produce a merge commit message
  interpret-trailers  Add or parse structured information in commit messages
  mailinfo            Extracts patch and authorship from a single e-mail message
  mailsplit           Simple UNIX mbox splitter program
  merge-one-file      The standard helper program to use with git-merge-index
  patch-id            Compute unique ID for a patch
  sh-i18n             Git's i18n setup code for shell scripts
  sh-setup            Common Git shell script setup code
  stripspace          Remove unnecessary whitespace

External commands
  askyesno
  credential-helper-selector
  flow
  lfs
```

Note:

If you're stuck in list view, press SHIFT + G to jump to the end and then q to exit.

Show the possible options for the status command in command line:

git status -help

Working with Git Branches

What is a Branch?

- A branch in Git is a separate version of the main repository.

Scenario: Updating Design in a Large Project

Without Git:

1. File Copies: Make copies of all relevant files to protect the live version.

2. Design Changes: Start updating the design, discovering dependencies with other files that also require changes.

3. Dependent Files: Copy dependent files, ensuring correct file references.

4. Emergency Fix: Encounter an unrelated error needing urgent attention.

5. File Management: Save all working files, noting their names.

6. Error Correction: Resolve the unrelated error and update the code.

7. Return to Design: Go back to the design work and complete it.

8. Finalizing Updates: Copy or rename files to publish the updated design.

9. Realization: Later, discover the unrelated error wasn't fixed in the new design due to prior copying.

With Git:

1. Branch Creation: Create a new branch called "new-design" to edit code without affecting the main branch.

2. Emergency Fix: Encounter an unrelated error needing urgent resolution.

3. New Fix Branch: Create a branch "small-error-fix" from the main project.

4. Fix Implementation: Correct the unrelated error and merge this branch with the main branch.

5. Complete Design Work: Return to the "new-design" branch and finish the updates.

6. Merge Changes: Merge the "new-design" branch with the main branch, integrating the earlier fix.

7. Branching Benefits: Work on different project parts simultaneously without main branch disruption.

8. Merging Nodes: Merge branches when work is complete.

9. Easy Switching: Swiftly switch between branches for various tasks.

10. Lightweight System: Branching in Git is efficient and fast.

New Git Branch

Work in a local repository to avoid impacting the main project.

git branch hello-world-images

Let's confirm that we have created a new branch:

git branch

  hello-world-images

* master

Explanation:

The "hello-world-images" branch is visible, with an asterisk (*) indicating the current branch is "master."

The "checkout" command switches to a specified branch in Git.

git checkout hello-world-images

Switched to branch 'hello-world-images'

Changes and a new file were added in the working directory of the main branch.

Now check the status of the current branch:

git status

On branch hello-world-images

Changes not staged for commit:

  (use "git add ..." to update what will be committed)

  (use "git restore ..." to discard changes in working directory)

        modified: index.html

Untracked files:

  (use "git add ..." to include in what will be committed)

        img_hello_world.jpg

no changes added to commit (use "git add" and/or "git commit -a")

Changes to index.html are unstage, and img_hello_world.jpg is untracked.

So we need to add both files to the Staging Environment for this branch:

git add --all

The `--all` option stages all changed files: new, modified, and deleted.

Check the status of the branch:

git status

On branch hello-world-images

Changes to be committed:

  (use "git restore --staged ..." to unstage)

    new file: img_hello_world.jpg

    modified: index.html

So we will commit them to the branch:

git commit -m "Added image to Hello World"

[hello-world-images 0312c55] Added image to Hello World

2 files changed, 1 insertion(+)

create mode 100644 img_hello_world.jpg

A new branch has been created, differing from the master branch.

Note:

The `-b` option in checkout creates and switches to a new branch if it doesn’t exist.

Switching Between Branches

On the hello-world-images branch, an image was added, prompting a file list.

ls

README.md bluestyle.css img_hello_world.jpg index.html

The new file img_hello_world.jpg is added, and the HTML file is updated correctly.

Now, let's see what happens when we change branch to master:

git checkout master

Switched to branch 'master'

The new image is not a part of this branch. List the files in the current directory again:

ls

README.md bluestyle.css index.html

img_hello_world.jpg is missing, and the HTML code has reverted.

Emergency Branch

git checkout -b emergency-fix

Switched to a new branch 'emergency-fix'

We have made changes in this file, and we need to get those changes to the master branch.
Check the status:

git status

On branch emergency-fix
Changes not staged for commit:
  (use "git add ..." to update what will be committed)
  (use "git restore ..." to discard changes in working directory)
        modified: index.html
no changes added to commit (use "git add" and/or "git commit -a")

stage the file, and commit:

git add index.html
git commit -m "updated index.html with emergency fix"

[emergency-fix dfa79db] updated index.html with emergency fix

1 file changed, 1 insertion(+), 1 deletion(-)

List the existing branches:

git branch

Move to the hello-world-images branch:

git checkout hello-world-images

Create, and move to a new branch with the name hello-you:

git checkout -b hello-you

Git Branch Merge

Merge Branches

First, we need to change to the master branch:

git checkout master

Switched to branch 'master'

Now we merge the current branch (master) with emergency-fix:

git merge emergency-fix

Updating 09f4acd..dfa79db

Fast-forward

index.html | 2 +-

1 file changed, 1 insertion(+), 1 deletion(-)

The emergency-fix branch can fast-forward to master, pointing both to the same commit.

Emergency-fix can be deleted as it is now identical to master, so:

git branch -d emergency-fix

Deleted branch emergency-fix (was dfa79db).

Merge Conflict

Move to hello-world-images to add img_hello_git.jpg and update index.html, so it shows it:

git checkout hello-world-images

Switched to branch 'hello-world-images'

commit for this branch:

git add --all

git commit -m "added new image"

[hello-world-images 1f1584e] added new image

2 files changed, 1 insertion(+)

create mode 100644 img_hello_git.jpg

Ready to merge hello-world-images into master after changes in index.html.

But what will happen to the changes we recently made in master?

```
git checkout master
git merge hello-world-images
```

```
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

The merge failed, as there is conflict between the versions for index.html. Let us check the status:

```
git status
```

```
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)
Changes to be committed:
        new file:   img_hello_git.jpg
        new file:   img_hello_world.jpg
Unmerged paths:
  (use "git add ..." to mark resolution)
        both modified:   index.html
```

Now we can stage index.html and check the status:

```
git add index.html
git status
```

On branch master

All conflicts fixed but you are still merging.

(use "git commit" to conclude merge)

Changes to be committed:

new file: img_hello_git.jpg

new file: img_hello_world.jpg

modified: index.html

The conflict has been fixed, and we can use commit to conclude the merge:

git commit -m "merged with hello-world-images after fixing conflicts"

[master e0b6038] merged with hello-world-images after fixing conflicts

And delete the hello-world-images branch:

git branch -d hello-world-images

Deleted branch hello-world-images (was 1f1584e).

Git GitHub Getting Started

GitHub Account

Create a Repository on GitHub

open the GitHub account then go the repository then click the + icon on the right corner of the GitHub then pressed the new repository then set a unique name of your repository then below the page and pressed the button create repository and then the work is completed.

Push Local Repository to GitHub9

…or create a new repository on the command line

```
echo "# All-In-One-University" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/RR0327/All-In-One-University.git
git push -u origin main
```

…or push an existing repository from the command line

```
git remote add origin https://github.com/RR0327/All-In-One-University.git
git branch -M main
git push -u origin main
```

Now we are going to push our master branch to the origin url, and set it as the default remote branch:

```
git push --set-upstream origin master
```

Push local master to origin and set it as the default upstream branch.

Note: First connection to GitHub requires authentication notification.

Git GitHub Edit Code

Edit Code in GitHub

Edit README.md in GitHub, commit changes to master, and add a commit description.

Git Pull from GitHub

Pulling to Keep up-to-date with Changes

In team projects, use Git's `pull` to fetch and merge the latest changes to stay updated.

Git Fetch

fetch gets all the change history of a tracked branch/repo.

on GitHub:

git fetch origin

remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 733 bytes | 3.00 KiB/s, done.
From https://github.com/w3schools-test/hello-world
  e0b6038..d29d69f master   -> origin/master

Now that we have the recent changes, we can check our status:

git status

On branch master

Your branch is behind 'origin/master' by 1 commit, and can be fast-forwarded.

(use "git pull" to update your local branch)

nothing to commit, working tree clean

Local branch is 1 commit behind origin/master, likely the updated README.md.

But lets double check by viewing the log:

git log origin/master

commit d29d69ffe2ee9e6df6fa0d313bb0592b50f3b853 (origin/master)

Author: w3schools-test <77673807+w3schools-test@users.noreply.github.com>

Date: Fri Mar 26 14:59:14 2021 +0100

Updated README.md with a line about GitHub

commit e0b6038b1345e50aca8885d8fd322fc0e5765c3b (HEAD -> master)

Merge: dfa79db 1f1584e

Author: w3schools-test

Date: Fri Mar 26 12:42:56 2021 +0100

merged with hello-world-images after fixing conflicts

...

...

Verify differences between local master and origin/master:

git diff origin/master

diff --git a/README.md b/README.md

index 23a0122..a980c39 100644

--- a/README.md

+++ b/README.md

@@ -2,6 +2,4 @@

Hello World repository for Git tutorial

This is an example repository for the Git tutoial on https://www.w3schools.com

-This repository is built step by step in the tutorial.

-

-It now includes steps for GitHub

+This repository is built step by step in the tutorial.

\ No newline at end of file

That looks precisely as expected! Now we can safely merge.

Git Merge

merge combines the current branch, with a specified branch.

Updates confirmed; merge current branch (master) with origin/master:

git merge origin/master

Updating e0b6038..d29d69f

Fast-forward

README.md | 4 +++-

1 file changed, 3 insertions(+), 1 deletion(-)

Check our status again to confirm we are up to date:

git status

On branch master

Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean

There! Your local git is up to date!

Git Pull

Use `pull` to fetch and merge remote changes into your branch.

Make another change to the Readme.md file on GitHub.

Use pull to update our local Git:

git pull origin

remote: Enumerating objects: 5, done.

remote: Counting objects: 100% (5/5), done.

remote: Compressing objects: 100% (3/3), done.

remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0

Unpacking objects: 100% (3/3), 794 bytes | 1024 bytes/s, done.

From https://github.com/w3schools-test/hello-world

  a7cdd4b..ab6b4ed master   -> origin/master

Updating a7cdd4b..ab6b4ed

Fast-forward

 README.md | 2 ++

 1 file changed, 2 insertions(+)

Use commands to update local Git.

Get all the change history of the origin for this branch:

git fetch origin

Git Push to GitHub

git commit -a -m "Updated index.html. Resized image"

[master e7de78f] Updated index.html. Resized image
 1 file changed, 1 insertion(+), 1 deletion(-)

check status:

git status

On branch master
Your branch is ahead of 'origin/master' by 1 commit.
  (use "git push" to publish your local commits)
nothing to commit, working tree clean

Now push our changes to our remote origin:

git push origin

Enumerating objects: 9, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 16 threads

Compressing objects: 100% (5/5), done.

Writing objects: 100% (5/5), 578 bytes | 578.00 KiB/s, done.

Total 5 (delta 3), reused 0 (delta 0), pack-reused 0

remote: Resolving deltas: 100% (3/3), completed with 3 local objects.

To https://github.com/w3schools-test/hello-world.git

  5a04b6f..facaeae master -> master

Git GitHub Branch

Create a New Branch on GitHub

On GitHub, access your repo, click "master," name the new branch, and click "Create branch."

The new branch is active, shown by the button now displaying "html-skeleton."

Edit the "index.html" file in the branch to start working.

Click "Preview changes" to highlight and view your edits.

Add a comment and click "Commit changes" to update the new branch on GitHub.

Git Pull Branch from GitHub

Pulling a Branch from GitHub

Now continue working on our new branch in our local Git.

Lets pull from our GitHub repository again so that our code is up-to-date:

git pull

remote: Enumerating objects: 5, done.

remote: Counting objects: 100% (5/5), done.

remote: Compressing objects: 100% (3/3), done.

```
remote: Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 851 bytes | 9.00 KiB/s, done.
From https://github.com/w3schools-test/hello-world
 * [new branch]      html-skeleton -> origin/html-skeleton
Already up to date.
```

Do a quick status check:

```
git status
```

```
On branch master
Your branch is up to date with 'origin/master'.
nothing to commit, working tree clean
```

And confirm which branches we have, and where we are working at the moment:

```
git branch
```

```
* master
```

we can use the -a option to see all local and remote branches:

```
git branch -a
```

```
* master
  remotes/origin/html-skeleton
  remotes/origin/master
```

Note: branch -r is for remote branches only.

We see that the branch html-skeleton is available remotely, but not on our local git. Lets check it out:

git checkout html-skeleton

Switched to a new branch 'html-skeleton'

Branch 'html-skeleton' set up to track remote branch 'html-skeleton' from 'origin'.

And check if it is all up to date:

git pull

Already up to date.

Identify the current branches and the active working branch.

git branch

* html-skeleton

  master

Open your editor to confirm that the GitHub branch changes have been pulled locally.

Git Push Branch to GitHub

Push a Branch to GitHub

Create a new local branch and push it to GitHub as before.

git checkout -b update-readme

Switched to a new branch 'update-readme'

Add a line to README.md and check the current branch status.

git status

On branch update-readme
Changes not staged for commit:
  (use "git add ..." to update what will be committed)
  (use "git restore ..." to discard changes in working directory)
        modified: README.md
no changes added to commit (use "git add" and/or "git commit -a")

We see that README.md is modified but not added to the Staging Environment:

git add README.md

Check the status of the branch:

git status

On branch update-readme
Changes to be committed:
  (use "git restore --staged ..." to unstage)
        modified: README.md

We are happy with our changes. So we will commit them to the branch:

```
git commit -m "Updated readme for GitHub Branches"

[update-readme 836e5bf] Updated readme for GitHub Branches
 1 file changed, 1 insertion(+)
```

Now push the branch from our local Git repository, to GitHub, where everyone can see the changes:

```
git push origin update-readme

Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 366 bytes | 366.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'update-readme' on GitHub by visiting:
remote:      https://github.com/w3schools-test/hello-world/pull/new/update-readme
remote:
To https://github.com/w3schools-test/hello-world.git
 * [new branch]      update-readme -> update-readme
```

Go to GitHub, and confirm that the repository has a new branch:

View changes in GitHub and merge into the master branch if approved.

Comparison shows changes from update-readme and html-skeleton, created from html-skeleton.

Git GitHub Flow

Working using the GitHub Flow

Learn GitHub flow: branch, commit, pull request, review, deploy, merge for effective collaboration.

Create a New Branch

Branching in Git keeps the master branch deployable; create a new branch for experiments before merging.

Note: Use descriptive names for branches to enhance team understanding and collaboration.

Make Changes and Add Commits

Make changes on the branch, commit with descriptive messages to track progress, and create a history for reversion.

Note: Use clear commit messages to explain changes and aid tracking for everyone involved.

Open a Pull Request

Pull requests notify others of changes for review and facilitate merging contributions into their branch.

Review

Pull requests enable collaborative reviews, discussions, and improvements through new commits based on feedback.

Note: GitHub shows new commit and feedback in the "unified Pull Request view".

Deploy

After approval, deploy the pull request for final testing; issues can be resolved by redeploying the master branch.

Note: Teams often have dedicated testing environments used for deploying branches.

Merge

Merge code into the master branch after testing, with pull requests documenting changes and decisions.

Note: You can add keywords to your pull request for easier searching!

Git GitHub Pages

Create a New Repository

Name the repository as your GitHub username followed by .github.io for it to function as a GitHub page.

Push Local Repository to GitHub Pages

Add the new repository as a remote named "gh-page" for GitHub Pages.

Copy the URL And add it as a new remote:

git remote add gh-page https://github.com/w3schools-test/w3schools-test.github.io.git

Make sure you are on the master branch, then push the master branch to the new remote:

git push gh-page master

Enumerating objects: 33, done.

Counting objects: 100% (33/33), done.

Delta compression using up to 16 threads

Compressing objects: 100% (33/33), done.

Writing objects: 100% (33/33), 94.79 KiB | 15.80 MiB/s, done.

Total 33 (delta 18), reused 0 (delta 0), pack-reused 0

remote: Resolving deltas: 100% (18/18), done.

To https://github.com/w3schools-test/w3schools-test.github.io.git

 * [new branch]   master -> master

Note: If this is the first time you are connecting to GitHub, you will get some kind of notification to authenticate this connection.

Git GitHub Fork

Add to Someone Else's Repository

Fork a Repository

A fork is a copy of a repository for contributing to or starting projects, available on GitHub and similar platforms.

Git Clone from GitHub

Clone a Fork from GitHub

Now we have our own fork, but only on GitHub. We also want a clone on our local Git to keep working on it.

A clone is a full copy of a repository, including all logging and versions of files.

Move back to the original repository, and click the green "Code" button to get the URL to clone:

Open your Git bash and clone the repository:

git clone https://github.com/w3schools-test/w3schools-test.github.io.git

Cloning into 'w3schools-test.github.io'...

remote: Enumerating objects: 33, done.

remote: Counting objects: 100% (33/33), done.

remote: Compressing objects: 100% (15/15), done.

remote: Total 33 (delta 18), reused 33 (delta 18), pack-reused 0

Receiving objects: 100% (33/33), 94.79 KiB | 3.16 MiB/s, done.

Resolving deltas: 100% (18/18), done.

Take a look in your file system, and you will see a new directory named after the cloned project:

ls

w3schools-test.github.io/

Note: To specify a specific folder to clone to, add the name of the folder after the repository URL, like this: git clone https://github.com/w3schools-test/w3schools-test.github.io.git myfolder

Navigate to the new directory, and check the status:

```
cd w3schools-test.github.io
git status
```

```
On branch master
Your branch is up to date with 'origin/master'.
nothing to commit, working tree clean
```

And check the log to confirm that we have the full repository data:

```
git log
```

```
commit facaeae8fd87dcb63629f108f401aa9c3614d4e6 (HEAD -> master, origin/master, origin/HEAD)
Merge: e7de78f 5a04b6f
Author: w3schools-test
Date:   Fri Mar 26 15:44:10 2021 +0100
    Merge branch 'master' of https://github.com/w3schools-test/hello-world
commit e7de78fdefdda51f6f961829fcbdf197e9b926b6
Author: w3schools-test
Date:   Fri Mar 26 15:37:22 2021 +0100
    Updated index.html. Resized image
.....
```

Configuring Remotes

Basically, we have a full copy of a repository, whose origin we are not allowed to make changes to.

Let's see how the remotes of this Git is set up:

```
git remote -v

origin  https://github.com/w3schools-test/w3schools-test.github.io.git (fetch)
origin  https://github.com/w3schools-test/w3schools-test.github.io.git (push)
```

We see that origin is set up to the original "w3schools-test" repository, we also want to add our own fork.

First, we rename the original origin remote:

```
git remote rename origin upstream
git remote -v

upstream        https://github.com/w3schools-test/w3schools-test.github.io.git (fetch)
upstream        https://github.com/w3schools-test/w3schools-test.github.io.git (push)
```

Then fetch the URL of our own fork.

And add that as origin:

```
git remote add origin https://github.com/kaijim/w3schools-test.github.io.git
git remote -v

origin  https://github.com/kaijim/w3schools-test.github.io.git (fetch)
origin  https://github.com/kaijim/w3schools-test.github.io.git (push)
upstream        https://github.com/w3schools-test/w3schools-test.github.io.git (fetch)
upstream        https://github.com/w3schools-test/w3schools-test.github.io.git (push)
```

Note: According to Git naming conventions, it is recommended to name your own repository origin, and the one you forked for upstream

Now we have 2 remotes:

- origin - our own fork, where we have read and write access
- upstream - the original, where we have read-only access

Rename the origin remote to upstream:

git remote rename origin upstream

Git GitHub Send Pull Request

Push Changes to Our GitHub Fork

We have made a lot of changes to our local Git.

Now we push them to our GitHub fork:

commit the changes:

git push origin

Enumerating objects: 8, done.

Counting objects: 100% (8/8), done.

Delta compression using up to 16 threads

Compressing objects: 100% (5/5), done.

Writing objects: 100% (5/5), 393.96 KiB | 32.83 MiB/s, done.

Total 5 (delta 0), reused 0 (delta 0), pack-reused 0

To https://github.com/kaijim/w3schools-test.github.io.git

  facaeae..ebb1a5c master -> master

Go to GitHub, and we see that the repository has a new commit.

Git Ignore and .gitignore

Git Ignore

Git uses a `.gitignore` file to exclude specific files, like logs and personal data, from tracking while keeping the `.gitignore` file itself tracked.

Create .gitignore

To create a .gitignore file, go to the root of your local Git, and create it:

touch .gitignore

Now open the file using a text editor.

We are just going to add two simple rules:

- Ignore any files with the .log extension
- Ignore everything in any directory named temp

Now all .log files and anything in temp folders will be ignored by Git.

Note: A single `.gitignore` applies to the whole repository, while additional `.gitignore` files in subdirectories target specific files within those directories.

Rules for .gitignore

Here are the general rules for matching patterns in .gitignore files:

Rules for .gitignore

- Blank Lines: Ignored.

- Comments: Lines starting with # are ignored.

- name: Matches all name files and folders (for example, /name.log, /name/file.txt, /lib/name.log).

- name/: Matches all files and folders in any name folder (for example, /name/file.txt, /name/log/name.log).

- no match: /name.log.

- name.file: Matches all files with the name.file (for example, /name.file, /lib/name.file).

- /name.file: matches only files in the root folder; does not match /lib/name.file.

- no match: /lib/name.file

- lib/name.file: Patterns specifying files in specific folders are always relative to root (even if you do not start with /) (for example, /lib/name.file).

- no match: name.file, /test/lib/name.file.

- **lib/name.file: Starting with ** before / specifies that it matches any folder in the repository, not just on root (for example, /lib/name.file, /test/lib/name.file).

- **name: Matches all name folders and files in any name folder (for example, /name/log.file, /lib/name/log.file, /name/lib/log.file).

- lib/**name: Matches all name folders and files in any name folder within the lib folder (for example, /lib/name/log.file, /lib/test/name/log.file, /lib/test/ver1/name/log.file).

- no match: /name/log.file.

- **.file: Matches all files withe .file extension (for example, /name.file, /lib/name.file).

- **name/: Matches all folders ending with name (for example, /lastname/log.file, /firstname/log.file).

- name?.file: ? matches a single non-specific character (for example, /names.file, /name1.file).

- no match: /names1.file

- name[a-z].file: [range] matches a single character in the specified range (including numeric characters) (for example, /names.file, /nameb.file).

- no match: /name1.file.

- name[abc].file: [set] matches a single character in the specified set of characters (for example, /namea.file, /nameb.file).

- no match: /names.file.

- name[!abc].file: [!set] matches a single character except those specified in the set (for example, /names.file, /namex.file).

- no match: /namesb.file.

- *.file: Matches all files withe .file extension (for example, /name.file, /lib/name.file).

- name/

!name/secret.log: Matches all files and folders in any name folder.

- ! specifies a negation or exception that matches all files and folders in any name folder, except name/secret.log (for example, /name/file.txt, /name/log/name.log).

- no match: /name/secret.log.

- *.file

!name.file: ! specifies a negation or exception. All files withe .file extention, except name.file (for example, /log.file, /lastname.file).

- no match: /name.file.

*.file

!name/*.file

junk.*: Adding new patterns after a negation will re-ignore a previous negated file. All files withe .file extention, except the ones in name folder. Unless the file name is junk.

(Example: /log.file, /name/log.file

- no match: /name/junk.file

Local and Personal Git Ignore Rules

Use .git/info/exclude to ignore files without sharing in the distributed .gitignore.

In .gitignore add a line to ignore all .temp files:

*.temp

In .gitignore add a line to ignore all files in any directory named temp:

temp/

In .gitignore add a single line to ignore all files named temp1.log, temp2.log, and temp3.log:

temp?.log

In .gitignore, ignore all .log files, except main.log:

*.log

!main.log

Git Security SSH

Git Security

Use HTTPS for repositories; switch to SSH for unsecured networks or project requirements.

What is SSH?

SSH is a secure shell protocol for remote access and file transfer, using public and private keys for encryption.

Generating an SSH Key Pair

In the command line for Linux, Apple, and in the Git Bash for Windows, you can generate an SSH key.

Let's go through it, step by step.

Start by creating a new key, using your email as a label:

ssh-keygen -t rsa -b 4096 -C "test@w3schools.com"

```
Generating public/private rsa key pair.
Enter file in which to save the key (/Users/user/.ssh/id_rsa):
Created directory '/Users/user/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /Users/user/.ssh/id_rsa
Your public key has been saved in /Users/user/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:********************************************** test@w3schools.com
The key's randomart image is:
+---[RSA 4096]----+
|                 |
|                 |
|                 |
|                 |
|                 |
|                 |
|                 |
|                 |
|                 |
+----[SHA256]-----+
```

You will be prompted with the following through this creation:

```
Enter file in which to save the key (/c/Users/user/.ssh/id_rsa):
```

Select a file location, or press "Enter" to use the default file location.

```
Enter passphrase (empty for no passphrase):
```

Enter same passphrase again:

Now we add this SSH key pair to the SSH-Agent (using the file location from above):

ssh-add /Users/user/.ssh/id_rsa

Enter passphrase for /Users/user/.ssh/id_rsa:

Identity added: /Users/user/.ssh/id_rsa (test@w3schools.com)

You will be prompted to supply the passphrase, if you added one.

Now the SSH key pair is ready to use.

NOTE:

• In this context, a **passphrase** is like a password, but longer, used to protect your SSH key. It's an extra secret that someone needs *in addition* to the key itself to use it. Think of it as a second lock on your door.

• A passphrase secures SSH keys, requiring input for use and preventing unauthorized access.

• ssh-keygen -t rsa -b 4096 -C "test@w3schools.com"

= The command generates a 4096-bit RSA SSH key pair with the comment "test@w3schools.com" for identification and prompts for a save location and optional passphrase.

• This command, `ssh-add /Users/user/.ssh/id_rsa`, adds your private SSH key (`/Users/user/.ssh/id_rsa`) to the SSH agent. The SSH agent then remembers your key's passphrase so you don't have to enter it every time you connect to a server using SSH. The context shows that after running the command, you were prompted for the passphrase, entered it, and the key was successfully added to the agent. Now, SSH connections using this key will be automatic (unless you reboot or remove the key from the agent).

Git GitHub Add SSH

Copy the SSH Public Key

Now we will use the clip < command to copy the public key to our clipboard:

clip < /Users/user/.ssh/id_rsa.pub

then,

1. Go to GitHub and click your profile picture in the top left corner, then select **Settings**.

2. In the settings menu, choose **SSH and GPG keys** and click the **New SSH key** button.

3. Enter a title for the key and paste your public SSH key into the **Key** field, then click **Add SSH Key**.

4. You'll be prompted to enter your GitHub password.

5. Your new SSH key will then be added.

Shortly:

Add a new SSH key on GitHub by navigating to Settings, entering a title, pasting the public key, and confirming with your password.

Test SSH Connection to GitHub

Now we can test our connection via SSH to GitHub:

ssh -T git@github.com

The authenticity of host 'github.com (140.82.121.3)' can't be established.

RSA key fingerprint is SHA256:nThbg6kXUpJWGl7E1IGOCspRomTxdCARLviKw6E5SY8.

Are you sure you want to continue connecting (yes/no/[fingerprint])? yes

Warning: Permanently added 'github.com,140.82.121.3' (RSA) to the list of known hosts.

Hi w3schools-test! You've successfully authenticated, but GitHub does not provide shell access.

- If the last line contains your username on GitHub, you are successfully authenticated!

Add New GitHub SSH Remote

To add a new SSH remote to Git, first, retrieve the SSH address from your GitHub repository.

shortly: Retrieve the SSH address from GitHub to add a new SSH remote to Git.

Then use that address to add a new origin:

git remote add ssh-origin git@github.com:w3schools-test/hello-world.git

Note: You can change a remote origin from HTTPS to SSH with the command: git remote set-url remote-name git@github.com:username/repository.git

This note explains how to switch the way your local Git repository connects to a remote repository (like one on GitHub) from using HTTPS to using SSH.

Here's a breakdown:

* **Remote origin:** Refers to the remote repository (usually on a service like GitHub) that your local Git repository is linked to. Think of it as the "original" or "central" repository your project is mirrored to. "origin" is often the default name for this remote.

* **HTTPS vs. SSH:** These are two different ways to authenticate with the remote repository.

  * **HTTPS (Hypertext Transfer Protocol Secure):** Uses your username and password (or a personal access token) for authentication. It's generally easier to set up initially.

* **SSH (Secure Shell):** Uses cryptographic keys (a private key on your computer and a corresponding public key on the remote service) for authentication. It's generally considered more secure and often eliminates the need to repeatedly enter credentials.

* **`git remote set-url remote-name git@github.com:username/repository.git`:** This is the command you use to change the connection method. Let's break it down:

* `git remote set-url`: This is the Git command to change the URL of a remote.

* `remote-name`: This is the name of the remote you want to change. It's usually "origin".

* `git@github.com:username/repository.git`: This is the SSH URL of your repository. It's important that you replace:

* `username` with your GitHub username.

* `repository` with the name of your repository.

In short, this command allows you to change your Git repository from using a potentially less secure HTTPS connection to a more secure SSH connection for pushing and pulling code. You'll need to have SSH keys set up on your computer and added to your GitHub account for this to work.

Shortly:

Switch your Git repository from HTTPS to SSH for better security using the command `git remote set-url origin git@github.com:username/repository.git`.

Example:

git remote set-url origin git@github.com:w3schools-test/hello-world.git

Exercise:

Add a new remote named ssh-origin connecting to x/y.git on abc.com using SSH:

git remote add ssh-origin git@abc.com:x/y.git

Replace the remote URL for origin with x/y.git on abc.com using SSH:

git remote set-url origin git@abc.com:x/y.git

Git Revert

revert is the command we use when we want to take a previous commit and add it as a new commit, keeping the log intact.

Shortly: The `revert` command creates a new commit that undoes a previous commit, preserving the commit history.

- Identify the previous commit and use `git revert` to create a new commit that undoes its changes.

Let's make a new commit, where we have "accidentally" deleted a file:

git commit -m "Just a regular update, definitely no accidents here..."

[master 16a6f19] Just a regular update, definitely no accidents here...

1 file changed, 0 insertions(+), 0 deletions(-)

delete mode 100644 img_hello_git.jpg

Now we have a part in our commit history we want to go back to. Let's try and do that with revert.

Git Revert Find Commit in Log

First thing, we need to find the point we want to return to. To do that, we need to go through the log.

To avoid the very long log list, we are going to use the --oneline option, which gives just one line per commit showing:

• The first seven characters of the commit hash

• the commit message

Shortly:

Use `git log --oneline` to view commits as one line each, showing the first seven characters of the hash and the commit message.

So let's find the point we want to revert:

git log --oneline

52418f7 (HEAD -> master) Just a regular update, definitely no accidents here...

9a9add8 (origin/master) Added .gitignore

81912ba Corrected spelling error

3fdaa5b Merge pull request #1 from w3schools-test/update-readme

836e5bf (origin/update-readme, update-readme) Updated readme for GitHub Branches

daf4f7c (origin/html-skeleton, html-skeleton) Updated index.html with basic meta

facaeae (gh-page/master) Merge branch 'master' of https://github.com/w3schools-test/hello-world

e7de78f Updated index.html. Resized image

5a04b6f Updated README.md with a line about focus

d29d69f Updated README.md with a line about GitHub

e0b6038 merged with hello-world-images after fixing conflicts

1f1584e added new image

dfa79db updated index.html with emergency fix

0312c55 Added image to Hello World

09f4acd Updated index.html with a new line

221ec6e First release of Hello World!

We want to revert to the previous commit: 52418f7 (HEAD -> master) Just a regular update, definitely no accidents here..., and we see that it is the latest commit.

Git Revert HEAD

We revert the latest commit using git revert HEAD (revert the latest change, and then commit), adding the option --no-edit to skip the commit message editor (getting the default revert message):

git revert HEAD --no-edit

[master e56ba1f] Revert "Just a regular update, definitely no accidents here..."

Date: Thu Apr 22 10:50:13 2021 +0200

1 file changed, 0 insertions(+), 0 deletions(-)

create mode 100644 img_hello_git.jpg

Now let's check the log again:

git log --oneline

e56ba1f (HEAD -> master) Revert "Just a regular update, definitely no accidents here..."

52418f7 Just a regular update, definitely no accidents here...

9a9add8 (origin/master) Added .gitignore

81912ba Corrected spelling error

3fdaa5b Merge pull request #1 from w3schools-test/update-readme

836e5bf (origin/update-readme, update-readme) Updated readme for GitHub Branches

daf4f7c (origin/html-skeleton, html-skeleton) Updated index.html with basic meta

facaeae (gh-page/master) Merge branch 'master' of https://github.com/w3schools-test/hello-world

e7de78f Updated index.html. Resized image

5a04b6f Updated README.md with a line about focus

d29d69f Updated README.md with a line about GitHub

e0b6038 merged with hello-world-images after fixing conflicts

1f1584e added new image

dfa79db updated index.html with emergency fix

0312c55 Added image to Hello World

09f4acd Updated index.html with a new line

221ec6e First release of Hello World!

Note: To revert to earlier commits, use git revert HEAD~x (x being a number. 1 going back one more, 2 going back two more, etc.)

Shortly:

Use `git revert HEAD --no-edit` to revert the latest commit, or `git revert HEAD~x` to revert earlier commits.

Exercise:

revert the two last commits:

git revert HEAD~1

Git Reset

Git Reset

reset is the command we use when we want to move the repository back to a previous commit, discarding any changes made after that commit.

- The `reset` command reverts the repository to a previous commit, discarding later changes.

• Step 1: Identify the previous commit;

• Step 2: Use reset to revert to that commit.

Git Reset Find Commit in Log

First thing, we need to find the point we want to return to. To do that, we need to go through the log.

To avoid the very long log list, we are going to use the --oneline option, which gives just one line per commit showing:

• The first seven characters of the commit hash - this is what we need to refer to in our reset command.

• the commit message

So let's find the point we want to reset to:

git log --oneline

e56ba1f (HEAD -> master) Revert "Just a regular update, definitely no accidents here..."

52418f7 Just a regular update, definitely no accidents here...

9a9add8 (origin/master) Added .gitignore

81912ba Corrected spelling error

3fdaa5b Merge pull request #1 from w3schools-test/update-readme

836e5bf (origin/update-readme, update-readme) Updated readme for GitHub Branches

daf4f7c (origin/html-skeleton, html-skeleton) Updated index.html with basic meta

facaeae (gh-page/master) Merge branch 'master' of https://github.com/w3schools-test/hello-world

e7de78f Updated index.html. Resized image

5a04b6f Updated README.md with a line about focus

d29d69f Updated README.md with a line about GitHub

e0b6038 merged with hello-world-images after fixing conflicts

1f1584e added new image

dfa79db updated index.html with emergency fix

0312c55 Added image to Hello World

09f4acd Updated index.html with a new line

221ec6e First release of Hello World!

We want to return to the commit: 9a9add8 (origin/master) Added .gitignore, the last one before we started to mess with things.

Git Reset

We reset our repository back to the specific commit using git reset commithash (commithash being the first 7 characters of the commit hash we found in the log):

git reset 9a9add8

Now let's check the log again:

git log --oneline

9a9add8 (HEAD -> master, origin/master) Added .gitignore

81912ba Corrected spelling error

3fdaa5b Merge pull request #1 from w3schools-test/update-readme

836e5bf (origin/update-readme, update-readme) Updated readme for GitHub Branches

daf4f7c (origin/html-skeleton, html-skeleton) Updated index.html with basic meta

facaeae (gh-page/master) Merge branch 'master' of https://github.com/w3schools-test/hello-world

e7de78f Updated index.html. Resized image

5a04b6f Updated README.md with a line about focus

d29d69f Updated README.md with a line about GitHub

e0b6038 merged with hello-world-images after fixing conflicts

1f1584e added new image

dfa79db updated index.html with emergency fix

0312c55 Added image to Hello World

09f4acd Updated index.html with a new line

221ec6e First release of Hello World!

Warning: Messing with the commit history of a repository can be dangerous. It is usually ok to make these kinds of changes to your own local repository. However, you should avoid making changes that rewrite history to remote repositories, especially if others are working with them.

Shortly:

Warning: Changing commit history is risky; safe locally, but avoid on shared remote repos.

Git Undo Reset

Even though the commits are no longer showing up in the log, it is not removed from Git.

If you know the commit hash you can reset to it:

git reset e56ba1f

Now let's check the log again:

git log --oneline

e56ba1f (HEAD -> master) Revert "Just a regular update, definitely no accidents here..."

52418f7 Just a regular update, definitely no accidents here...

9a9add8 (origin/master) Added .gitignore

81912ba Corrected spelling error

3fdaa5b Merge pull request #1 from w3schools-test/update-readme

836e5bf (origin/update-readme, update-readme) Updated readme for GitHub Branches

daf4f7c (origin/html-skeleton, html-skeleton) Updated index.html with basic meta

facaeae (gh-page/master) Merge branch 'master' of https://github.com/w3schools-test/hello-world

e7de78f Updated index.html. Resized image

5a04b6f Updated README.md with a line about focus

d29d69f Updated README.md with a line about GitHub

e0b6038 merged with hello-world-images after fixing conflicts

1f1584e added new image

dfa79db updated index.html with emergency fix

0312c55 Added image to Hello World

09f4acd Updated index.html with a new line

221ec6e First release of Hello World!

Git Amend

Git commit --amend

commit --amend is used to modify the most recent commit.

It combines changes in the staging environment with the latest commit, and creates a new commit.

This new commit replaces the latest commit entirely.

Shortly:

`commit --amend` modifies the latest commit by adding staged changes and replacing the old commit.

Git Amend Commit Message

One of the simplest things you can do with --amend is to change a commit message.

Let's update the README.md and commit:

git commit -m "Adding plines to reddme"

[master 07c5bc5] Adding plines to reddme
1 file changed, 3 insertions(+), 1 deletion(-)

Now let's check the log:

git log --oneline

07c5bc5 (HEAD -> master) Adding plines to reddme
9a9add8 (origin/master) Added .gitignore
81912ba Corrected spelling error
3fdaa5b Merge pull request #1 from w3schools-test/update-readme
836e5bf (origin/update-readme, update-readme) Updated readme for GitHub Branches
daf4f7c (origin/html-skeleton, html-skeleton) Updated index.html with basic meta
facaeae (gh-page/master) Merge branch 'master' of https://github.com/w3schools-test/hello-world
e7de78f Updated index.html. Resized image
5a04b6f Updated README.md with a line about focus
d29d69f Updated README.md with a line about GitHub

e0b6038 merged with hello-world-images after fixing conflicts

1f1584e added new image

dfa79db updated index.html with emergency fix

0312c55 Added image to Hello World

09f4acd Updated index.html with a new line

221ec6e First release of Hello World!

Oh no! the commit message is full of spelling errors. Embarrassing. Let's amend that:

git commit --amend -m "Added lines to README.md"

[master eaa69ce] Added lines to README.md

Date: Thu Apr 22 12:18:52 2021 +0200

1 file changed, 3 insertions(+), 1 deletion(-))

And re-check the log:

git log --oneline

eaa69ce (HEAD -> master) Added lines to README.md

9a9add8 (origin/master) Added .gitignore

81912ba Corrected spelling error

3fdaa5b Merge pull request #1 from w3schools-test/update-readme

836e5bf (origin/update-readme, update-readme) Updated readme for GitHub Branches

daf4f7c (origin/html-skeleton, html-skeleton) Updated index.html with basic meta

facaeae (gh-page/master) Merge branch 'master' of https://github.com/w3schools-test/hello-world

e7de78f Updated index.html. Resized image

5a04b6f Updated README.md with a line about focus

d29d69f Updated README.md with a line about GitHub

e0b6038 merged with hello-world-images after fixing conflicts

1f1584e added new image

dfa79db updated index.html with emergency fix

0312c55 Added image to Hello World

09f4acd Updated index.html with a new line

221ec6e First release of Hello World!

We see the previous commit is replaced with our amended one!

Warning: Messing with the commit history of a repository can be dangerous. It is usually ok to make these kinds of changes to your own local repository. However, you should avoid making changes that rewrite history to remote repositories, especially if others are working with them.

Shorty:

Warning: Altering commit history is risky; safe in local repos, not in shared remote ones.

Git Amend Files

Adding files with --amend works the same way as above. Just add them to the staging environment before committing.

Git Quiz

Question 1:

What is Git?

A version control system. Your answer

A nickname for GitHub.

A remote repository platform.

A programming language.

Question 2:

Git is the same as GitHub.

False Your answer

True

Question 3:

What is the command to get the installed version of Git?

git --version Your answer

git help version

gitVersion

getGitVersion

Question 4:

Which option should you use to set the default user name for every repository on your computer?

--global Your answer

--all

--A

No need to specify, that is the default.

Question 5:

What is the command to set the user email for the current repository?

git config user.email Your answer

git config email

git email.user

Question 6:

What is the command to add all files and changes of the current folder to the staging environment of the Git repository?

git add Your answer

git add --files

git add --all Correct answer

Question 7:

What is the command to get the current status of the Git repository?

git status Your answer

git config --status

--status

git getStatus

Question 8:

What is the command to initialize Git on the current repository?

git init Your answer

initialize git

git start

start git

Question 9:

Git automatically adds new files to the repository and starts tracking them.

False Your answer

True

Question 10:

Git commit history is automatically deleted:

Commit history is never automatically deleted. Your answer

Every year.

Every 2 weeks.

Every month.

Question 11:

What is the command to commit the staged changes for the Git repository?

git commit Your answer

git snapshot

git com

git save

Question 12:

What is the command to commit with the message "New email":

git commit -m "New email" Your answer

git commit message "New email"

git commit -log "New email"

git commit -mess "New email"

Question 13:

What is the command to view the history of commits for the repository?

git log Your answer

git commits

git history

git --full-log

Question 14:

What is the command to see the available options for the commit command?

git commit -help Your answer

gitHelp commit

git commit readme

git commitHelp

Question 15:

In Git, a branch is:

A separate version of the main repository. Your answer

A secret part of Git config.

Nothing, it is a nonsense word.

A small wooden stick you can use to type commands.

Question 16:

What is the command to create a new branch named "new-email"?

git branch new-email Your answer

git add branch "new-email"

git newBranch "new-email"

git branch new "new-email"

Question 17:

What is the command to move to the branch named "new-email"?

git checkout new-email Your answer

git branch -move new-email

git checkout branch new-email

git branch new-email

Question 18:

What is the option, when moving to a branch, to create the branch it if it does not exist?

-b Your answer

-all

-new

-newbranch

Question 19:

What is the command to merge the current branch with the branch "new-email"?

git merge new-email Your answer

git add new-email

git commit -merge new-email

Question 20:

What is the command to delete the branch "new-email"

git branch -d new-email Your answer

git gone new-email

git delete branch new-email

git delete new-email

Question 21:

What is the command to add the remote repository "https://abc.xyz/d/e.git" as "origin"?

git remote add origin https://abc.xyz/d/e.git Your answer

git add origin https://abc.xyz/d/e.git

git remote https://abc.xyz/d/e.git

git origin=https://abc.xyz/d/e.git

Question 22:

What is the command to push the current repository to the remote origin?

git push origin Your answer

git remote push

git remote commit

git merge remote

Question 23:

What is the command to get all the change history of the remote repository "origin"?

git fetch origin Your answer

git origin help

git status remote origin

git get log origin

Question 24:

What is the command to show the differences between the current branch and the branch "new-email"?

git diff new-email Your answer

git changes new-email

git log new-email

git status new-email

Question 25:

Git Pull is a combination of:

fetch and merge Your answer

add and commit

branch and checkout

Explanation of the quiz:

1. Git is a version control system.
2. False, Git and GitHub are not the same.
3. Use `git --version` to check the installed Git version.
4. Use `--global` to set the default user name for all repositories.
5. Set the user email with `git config user.email`.
6. `git add --all` adds all files and changes to staging.
7. Use `git status` to check the repository's current status.
8. Initialize Git repository with `git init`.
9. False, Git does not automatically track new files.
10. Commit history is never automatically deleted.
11. Use `git commit` to commit staged changes.
12. To commit with a message, use `git commit -m "message"`.
13. View commit history with `git log`.
14. Use `git commit -help` to see commit options.
15. A branch is a separate version of the main repository.

16. Create a new branch with `git branch new-email`.
17. Move to a branch using `git checkout branch-name`.
18. Use `-b` to create a new branch if it doesn’t exist.
19. Merge branches with `git merge branch-name`.
20. Delete a branch with `git branch -d branch-name`.
21. Add a remote repository using `git remote add origin URL`.
22. Push changes to origin with `git push origin`.
23. Get change history from origin with `git fetch origin`.
24. Show differences with `git diff branch-name`.
25. Git Pull combines fetch and merge.

- **Whole Concept in shortly:**

Git and GitHub Concept Note

Git is a distributed version control system designed to track changes in code, manage collaboration, and maintain the full history of a project. GitHub is a cloud-based platform built on Git that offers tools for remote hosting, review, and deployment.

Why Use Git?

- Tracks code changes and associates them with specific contributors.
- Enhances team collaboration through branches and pull requests.
- Enables efficient version management and secure rollback capabilities.
- Offers storage-efficient change tracking.

What Git Does:

- Initializes repositories (`git init`) and tracks changes.
- Stages updates (`git add`) and commits them with messages (`git commit -m`).
- Clones repositories for local work and pushes updates to remotes.
- Supports branching for isolated workstreams and merging for collaboration.
- Handles merge conflicts and preserves history through commit logs.

Working with Git

- `git status`: See file tracking and changes.
- `git add index.html`: Stage a file.
- `git commit -m "message"`: Save staged changes.
- `git log --oneline`: Compact view of commit history.
- `git revert HEAD`: Revert latest commit.
- `git reset <commit_hash>`: Hard rollback to a previous state.
- `git branch <branch_name>`: Create a new branch.
- `git checkout <branch_name>`: Switch branches.
- `git merge <branch_name>`: Merge branches.
- `git push origin master`: Upload local changes to GitHub.

GitHub Basics

- Web interface for Git repositories.
- Allows code hosting, editing, and collaboration.
- Create repositories, manage issues, and track pull requests.
- Clone, push, and pull with GitHub URLs.
- GitHub Pages for hosting static sites.

Branches in Practice Branching is one of Git's most powerful features. Use branches to:

- Work on new features independently.
- Apply emergency fixes without disrupting main development.
- Safely test and experiment.
- Merge updates into the main branch once approved.

Working with Remotes

- `git remote add origin <URL>`: Link to GitHub.
- `git fetch origin`: Pull all updates from remote without merging.
- `git pull origin`: Fetch and merge changes.
- `git push origin`: Push changes to GitHub.

Security via SSH

- Use SSH for encrypted access to GitHub.
- Generate keys using `ssh-keygen` and add to GitHub.
- Authenticate securely without re-entering passwords.

.gitignore Essentials Use `.gitignore` to prevent tracking unwanted files:

- `*.log`: Ignores all log files.
- `temp/`: Ignores entire temp folders.
- `!main.log`: Exception to a rule.

Useful Git Tricks

- `git commit --amend`: Change last commit message or add staged changes.
- `git stash`: Temporarily save unfinished work.
- `git diff`: See changes between commits, branches, or working files.

GitHub Flow Overview

1. Create a branch.
2. Make changes and commit.
3. Open a pull request.
4. Review and discuss.
5. Merge to main.
6. Deploy.

Beginner Advantages

- Simple command-line commands.
- Immediate visual feedback using `git status`.
- Visual tools available (GitHub Desktop, VSCode).

Advanced Capabilities

- Full history management.
- Efficient branching and rebasing.
- Large project scaling and CI/CD support.
- Forking, upstream management, and complex merge resolutions.

Top Quiz Highlights for Practice

1. What is Git?
   A version control system.
2. Git is the same as GitHub.
   False.
3. Command to check Git version:
   `git --version`
4. Set global username:
   `git config --global user.name "Name"`
5. Add all changes to staging:
   `git add --all`
6. Commit staged changes:
   `git commit -m "Message"`
7. Initialize Git:
   `git init`
8. View commit log:
   `git log`
9. Create a branch:
   `git branch branch-name`
10. Merge a branch:
    `git merge branch-name`
11. Push local repository to GitHub:
    `git push origin branch-name`
12. Pull changes from GitHub:
    `git pull origin`
13. Add a remote repo:
    `git remote add origin URL`
14. Revert the last commit:
    `git revert HEAD`
15. What does Git Pull do?
    It combines fetch and merge.

- More Short:

Git and GitHub Concept Note

Git is a distributed version control system that helps developers track changes in source code efficiently. It enables seamless collaboration, ensures accountability, and maintains a complete project history. GitHub, a web-based platform, enhances Git by offering repository hosting, collaborative tools, and additional features for better code management.

Key Features of Git:

- Tracks code changes, authorship, and timestamps.
- Supports teamwork through branching and merging.
- Efficiently manages the entire project history in repositories.

Fundamental Git Operations:

- Initialize a Git repository with 'git init'.
- Stage changes using 'git add' and commit them with 'git commit'.
- View commit history via 'git log' and undo changes using 'git revert' or 'git reset'.
- Work on separate features using 'git branch' and merge updates with 'git merge'.
- Resolve merge conflicts manually and commit the resolved changes.

Essential Git Commands:

- git status: Check repository status.
- git add -A: Stage all modified files.
- git commit -m "message": Commit staged changes.
- git log --oneline: View a concise commit history.
- git diff: Compare file changes.
- git checkout branch-name: Switch branches.
- git branch -d branch-name: Delete a branch.

GitHub Integration:

- Hosts Git repositories online for collaboration and backup.
- Connect local repositories using 'git remote add origin [URL]'.
- Upload code with 'git push origin branch-name' and fetch updates using 'git pull origin'.
- Supports GitHub Pages for hosting static websites.
- Uses SSH keys for secure authentication and access.

Collaborative Development Workflow:

- Fork and clone repositories to work independently.

- Open pull requests to propose and review changes before merging.
- Follow GitHub Flow: Branch > Commit > Pull Request > Review > Merge > Deploy.
- Enables distributed teams to work in parallel with clear version control.

Advanced Git Techniques:

- Use .gitignore to exclude unnecessary files.
- Temporarily store work with 'git stash'.
- Clean up commit history with 'git rebase'.
- Modify past commits with reset and amend options.

Security and Authentication:

- SSH keys provide encrypted and password-free authentication.
- Securely manage repository access through GitHub settings.

Git and GitHub for Beginners:

- Simplified commands and clear outputs make Git easy to learn.
- Graphical interfaces like GitHub Desktop and VSCode extensions enhance usability.
- Structured workflows reduce errors and improve understanding.

Git and GitHub for Experts:

- Enables complex workflows, branching strategies, and automation pipelines.
- Provides detailed control over repository history and optimizations.
- Supports continuous integration, deployment, and large-scale project management.

Conclusion: Git and GitHub provide a structured and efficient way to manage code for projects of all sizes. Beginners benefit from simple commands and intuitive workflows, while experts can leverage advanced features for scalable and automated development. Mastering these tools ensures better collaboration, project tracking, and seamless deployment processes.